

Article

Adaptive Healthcare Monitoring Through Drift-Aware Edge-Cloud Intelligence

Aleksandra Stojnev Ilic ¹, Milos Ilic ², Natalija Stojanovic ¹ and Dragan Stojanovic ^{1,*}

¹ Department of Computer Science, Faculty of Electronic Engineering, University of Nis, 18104 Nis, Serbia; aleksandra.stojnev@elfak.ni.ac.rs (A.S.I.); natalija.stojanovic@elfak.ni.ac.rs (N.S.)

² Department of Computer Science, Toplica Academy of Applied Studies, 18400 Prokuplje, Serbia; milos.ilic@vpskp.edu.rs

* Correspondence: dragan.stojanovic@elfak.ni.ac.rs

Abstract

Continuous healthcare monitoring systems generate non-stationary physiological data streams, where evolving statistical properties and patterns often invalidate static models and fixed user classifications. To address this challenge, we propose drift-aware adaptive architecture that integrates concept drift detection into a distributed edge–cloud data analytics pipeline. In the proposed design, a concept drift is elevated from a maintenance signal to the primary mechanism governing user-state adaptation, model evolution, and inference consistency. Within the proposed system, the edge tier performs low-latency inference and preliminary drift screening under strict resource constraints, while the cloud tier executes advanced drift detection and validation, orchestrates user reclassification and model retraining, and manages model evolution. A feedback loop synchronizes edge and cloud operations, ensuring that detected drift triggers appropriate system transitions, either reassigning a user to an updated state category or initiating targeted model updates. This architecture reduces reliance on static group assignments, improves personalization, and preserves model fidelity under evolving physiological conditions. We analyze the drift types most relevant to healthcare data streams, evaluate the suitability of lightweight and cloud-grade drift detectors, and define the system requirements for stability, responsiveness, and clinical safety. Evaluation across 21 concurrent users demonstrates that drift-aware adaptation reduced prediction MAE by 40.6% relative to periodic retraining, with an end-to-end adaptation latency of 66 ± 37 s. Hierarchical cloud validation reduced the false-positive retraining rate from 88.9% (edge-only triggering) to 27.3%, while maintaining uninterrupted inference throughout all adaptation events.

Keywords: adaptive healthcare monitoring; drift-aware edge-cloud intelligence; edge-cloud computing; healthcare IoT; adaptive systems; distributed inference; model lifecycle management



Academic Editors: Dimitrios Dechouniotis and Ioannis Dimolitsas

Received: 10 February 2026

Revised: 14 March 2026

Accepted: 16 March 2026

Published: 17 March 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Modern healthcare monitoring systems increasingly depend on continuous streams of physiological data collected by wearable sensors. Continuous glucose monitoring (CGM) is a representative example, generating minute-level measurements that support near real-time anomaly detection, personalized state prediction, and long-term metabolic profiling. Applications that analyze the data generated by these sensors rely heavily on various static machine-learning models deployed in operational environments. However, the underlying

data distributions are rarely stationary. Physiological signals evolve in response to circadian patterns, meal structure, physical activity, stress, medication, sensor degradation, etc. As a result, the statistical relationship between inputs and outputs changes over time. This phenomenon, known as concept drift, poses a fundamental challenge to any system that must sustain accuracy and reliability during extended deployment.

In practice, concept drift is often not an exception or anomaly but a normal feature of biomedical data streams. Static models deteriorate quickly as they become misaligned with a user's evolving physiology or behavior. Periodic retraining on a fixed schedule is simple to set up, but it is often both inadequate and inefficient. It does not react quickly to abrupt drifts, it can spend computing resources when the model does not need to change, and it assumes that drift occurs on a regular, known schedule. For healthcare applications that require continuous high-quality analytics, concept drift should be handled as a regular part of day-to-day operations throughout the system life cycle, rather than as an occasional offline monitoring task.

The distributed nature of healthcare analytics underlines the need for drift-aware adaptive methods in systems for data analysis. Wearable-based decision support for healthcare applications is often implemented across a heterogeneous edge–cloud continuum, where sensing devices and near-edge gateways perform latency-critical preprocessing and inference, while cloud platforms are used for heavier computation, cross-user generalization, and long-term model management. Each tier introduces distinct constraints: edge devices operate under strict limitations in energy, memory, and compute resources, but are close to the data source and maintain low-latency responsiveness; cloud servers can scale resources and have access to global datasets but have higher communication overhead and depend on stable connectivity. Deciding where inference should run, where to detect drift, and where and when to update the model is therefore a core engineering challenge that directly affects system correctness, cost, and responsiveness.

A growing body of research explores concept drift detection algorithms, but many of these studies assume that processing happens in a centralized cloud setting or assess drift using offline evaluations. These approaches do not account for the complexity of evolving physiological data that are distributed across sources and change over time. Drift will appear first at the edge, long before aggregated and/or processed data are sent to the cloud. It can happen gradually or abruptly, and it may reflect not only degradation in model performance but also meaningful transitions in a user's metabolic state. Consequently, drift detection must be built directly into the operational pipeline and appropriately integrated with the mechanisms used for model updates and deployment management. Treating drift as a secondary diagnostic signal is insufficient for systems that must remain adaptive, stable, and resource-efficient under real-world constraints.

To achieve this integration, a continuous model lifecycle that spans both the edge and the cloud must be defined. Inference pipelines should be able to shift between tiers depending on latency requirements, resource availability, and the stability of the predicted output. Drift detection components must operate in a distributed manner, with lightweight statistical or uncertainty-based detectors at the edge and more computationally intensive reconstruction-based or model-based detectors in the cloud. The resulting drift signals should be routed into a feedback loop that orchestrates the adaptation process, including decisions such as when to recalibrate, when to retrain, when to override local models, and when to propagate new model versions. This process should balance timely updates against the risk of excessive retraining, since frequent retraining can destabilize the system and place unnecessary demands on cloud resources.

Developing a robust drift-aware edge–cloud architecture raises additional challenges in terms of model provisioning, version control, multi-user heterogeneity, and cost-optimal allocation of computing resources. Healthcare systems must handle users whose physiological patterns evolve at different timescales, exhibit different types of drift, or require distinct model configurations. Adaptation strategies that work well for one user may introduce unnecessary computations for another. Designing architecture-level mechanisms that interpret drift not only as a signal of model failure but also as an indicator of long-term physiological change introduces an additional layer of complexity. This dual role of drift as both an engineering challenge and a potential proxy for physiological transitions reinforces the need for a tightly integrated, fully automated adaptation workflow.

This paper addresses these challenges by proposing an adaptive, concept-drift-aware edge–cloud data analytics architecture for healthcare data streams, using CGM as a representative sensing modality. The proposed system combines distributed drift detection, cross-tier inference, and a closed feedback loop for model updates, so machine-learning models deployed at scale can adjust over time as data and conditions change. The architecture treats drift as a first-class system event that can trigger model refinement, retraining, or deployment of updated models tailored to a user’s evolving profile. Although CGM data serve as the main example, the architecture generalizes to a wide range of streaming biosensing scenarios, thus providing a general reference for future healthcare analytics systems where adaptivity, resilience, and distributed intelligence are fundamental design principles. This work focuses on the architectural and algorithmic aspects of drift-aware adaptation; considerations such as privacy-preserving data handling, regulatory compliance for clinical deployment, and security of the model distribution pipeline are beyond the current scope but represent important directions for future work.

This paper presents a drift-aware edge–cloud architecture for continuous inference over non-stationary physiological data streams, with a particular focus on safety-critical e-health monitoring scenarios, and makes the following contributions:

- A drift-aware edge–cloud architecture for continuous healthcare monitoring under non-stationary physiological data streams, incorporating a hierarchical drift-handling strategy that separates lightweight edge screening from cloud-based validation to prevent premature adaptations (Section 3).
- A closed-loop adaptation pipeline that automatically triggers retraining, validation, and redeployment of models in response to validated drift events, and supports a dynamic user–model alignment mechanism that mitigates unnecessary retraining by reassigning users to compatible model states after confirmed drift events (Section 3).
- Experimental evaluation on synthetic multi-user streams and real CGM data, demonstrating stable inference and reduced adaptation overhead in a containerized edge–cloud deployment (Section 4).

The proposed design is implemented as a containerized, event-driven prototype integrating drift detection, model lifecycle management, and edge deployment, and is evaluated using continuous glucose monitoring data to demonstrate end-to-end feasibility, stability, and adaptability under realistic non-stationary conditions.

The paper is organized as follows. Section 2 reviews related work on concept drift, edge–cloud healthcare systems, and adaptation strategies. Section 3 presents the proposed architecture and its prototype implementation, as well as the datasets used for evaluation and testing. Section 4 describes the experimental evaluation, obtained results, and known limitations. Section 5 concludes with future directions.

2. Related Work

The healthcare domain is increasingly adopting Internet of Medical Things (IoMT) technologies to support continuous and real-time patient monitoring [1]. While cloud-centric architectures have traditionally been employed to process and store large volumes of physiological data, their reliance on centralized computation introduces latency, bandwidth overhead, and privacy risks that are particularly problematic in time-critical healthcare scenarios [2]. To address these limitations, prior studies have proposed hybrid edge–cloud architectures that distribute computational tasks across edge nodes and cloud infrastructures, enabling low-latency analytics, improved data privacy, and more efficient network utilization [3]. Recent research has explored the integration of artificial intelligence techniques within edge–cloud healthcare systems and demonstrated their potential for real-time anomaly detection, early diagnosis, and personalized health interventions using data from wearable sensors and bedside monitoring devices [4,5]. In these systems, lightweight inference is typically performed at the edge to support immediate decision-making, while the cloud is leveraged for computationally intensive tasks such as model training, historical analysis, and population-level insight generation. Despite these advances, most existing edge–cloud healthcare monitoring approaches implicitly assume relatively stable data distributions.

Concept drift is a fundamental challenge in streaming machine learning, referring to temporal changes in data distributions or in the relationship between input features and target variables. Formally, drift occurs when the joint probability distribution $P(X, y)$ changes between time t and $t + \Delta t$ [6]. This temporal instability violates the assumption that training and deployment data follow the same distribution [7]. Concept drift is typically classified by its temporal behavior: sudden drift involves abrupt changes, gradual drift reflects transitional shifts, incremental drift denotes slow continuous evolution, and recurring drift describes periodically reappearing concepts [8]. Abdullahi et al. emphasize that temporal dependencies and autocorrelation in time-series data substantially increase the complexity of drift detection and adaptation compared to independent data streams [9].

Physiological data streams differ from generic concept drift settings due to their inherent non-stationarity, patient specificity, and real-time constraints. Signals fluctuate naturally with circadian rhythms, physical activity, emotional states, and behavioral patterns [10], complicating the distinction between expected variability and true drift that requires adaptation. Patient-specific characteristics limit cross-patient generalization [11], while strict latency requirements in healthcare monitoring constrain the computational complexity of drift detection and adaptation [12,13]. Clinical decisions further demand that drift methods balance sensitivity, robustness, interpretability, and efficiency, as false alarms or missed drifts can compromise patient safety [14]. Additionally, covariate shifts (changes in input feature distribution) and label drift (changes in health state prevalence) often coexist with concept drift, collectively threatening predictive accuracy.

Existing methods for concept drift detection in healthcare include, among others, statistical, entropy-based, sliding window, and ensemble approaches, each with trade-offs in sensitivity, computational cost, and real-time suitability. Statistical methods monitor deviations in data distributions or prediction errors [15], but may fail to detect gradual drift masked by physiological variability. Entropy-based methods capture signal complexity and structural changes without labeled data. They are sensitive to subtle changes but are computationally demanding. Sliding window approaches maintain recent data segments to detect drift, balancing responsiveness with sufficient data for reliable detection [16]; Desale et al. combine deep learning with adaptive windows [17]. Ensemble methods track disagreement or individual model performance across multiple models, offering robustness at the cost of higher complexity [18].

Several strategies have been proposed to adapt models once drift is detected. These include: model retraining, triggered selectively based on drift severity and often using sliding windows of recent data [19,20]; incremental/online learning, which continuously updates model parameters without full retraining [21]; ensemble adaptation, adjusting member weights or composition based on drift type and severity [22]; generative replay, synthesizing representative examples of previous concepts to maintain performance on recurring patterns [13]; and adaptive data buffering, retaining representative historical samples considering age, representativeness, and drift stability [20].

Healthcare monitoring systems increasingly rely on an edge–cloud continuum to support real-time analytics under stringent latency constraints. The edge tier, composed of resource-constrained IoT devices and wearables, handles local feature extraction, anomaly detection, and time-critical inference. The cloud tier supports model training, historical analysis, and knowledge distillation, where latency requirements are less strict [23]. While this hierarchical architecture enables task offloading and efficient resource use, existing implementations typically rely on static or periodically retrained models, leaving continuous drift-aware adaptation across tiers largely unexplored [24].

Recent benchmarking of unsupervised drift detectors on real-world data streams confirms that no single algorithm dominates across drift types, reinforcing the multi-detector ensemble strategy adopted in this work [25]. Complementary surveys on edge computing cost optimization further demonstrate that latency and resource expenditure are tightly coupled in distributed IoT deployments, underscoring the need for adaptive, drift-triggered retraining rather than fixed-schedule approaches [26].

Table 1 provides a comparative summary of representative prior works along five architectural dimensions relevant to drift-aware healthcare monitoring.

Table 1. Overview of representative prior studies and their architectural characteristic.

Reference	Distributed Detection	Drift-Triggered Adaptation	User Reclassification	Model Specialization	Evaluation Type
Zertal et al. [13]	No (centralized)	Yes (generative replay)	No	No	Offline dataset
Yang and Shami [16]	Partial (edge-only)	Yes (lightweight)	No	No	Simulation
Desale and Shindle [17]	No (centralized)	Yes (sliding window)	No	No	Offline dataset
Pratyusha and Sinha [18]	No (centralized)	Yes (ensemble weighting)	No	No	Offline dataset
Prashanth et al. [20]	No (centralized)	Yes (adaptive buffering)	No	Partial	Offline dataset
Toor et al. [22]	No (centralized)	Yes (adaptive)	No	No	Streaming simulation
Proposed system	Yes (hierarchical edge-cloud)	Yes (drift-triggered)	Yes	Supported by the architecture	Prototype testbed (synthetic + real)

Despite extensive research in concept drift detection and adaptation, prior work generally treats these mechanisms independently from edge–cloud deployment. Table 1 summarizes the positioning of representative systems along five key dimensions. Among the reviewed works, none simultaneously integrates (i) distributed, hierarchical drift detection spanning edge and cloud tiers, (ii) drift-triggered user reclassification and model specialization, (iii) automated model lifecycle management with retraining and redeployment, and (iv) a closed feedback loop from edge-level detection to cloud adaptation and back to edge deployment.

3. Adaptive Healthcare Monitoring System—Architecture and Prototype

3.1. Proposed System Architecture

To address the challenges of continuous inference using non-stationary physiological data streams, we propose a multi-tier edge–cloud architecture in which concept drift is treated as a first-class supervisory system signal. This feedback-based flow reflects the architectural differences in capabilities and responsibilities of each tier. The edge utilizes proximity to data sources and responsiveness under resource constraints, while the cloud provides aggregation, validation, and coordination using longer time windows and information from larger groups of users or devices. In this design, inference runs continuously and is assigned to the tier that best satisfies latency and resource constraints, while drift handling is distributed and hierarchical. Early indicators of drift are identified close to the data source, whereas confirmation and adaptation decisions are centralized. One of the main design principles of the architecture is also the strict separation between drift screening, drift validation, and adaptation decision-making, ensuring that resource-constrained components are never responsible for irreversible or computationally demanding system-level changes. Concept drift is treated as a system-level coordination signal that governs model lifecycle transitions and deployment behavior across the edge–cloud continuum, and is used to explicitly balance responsiveness to evolving data distributions against robustness to transient noise. A high-level overview of the system architecture is shown in Figure 1.

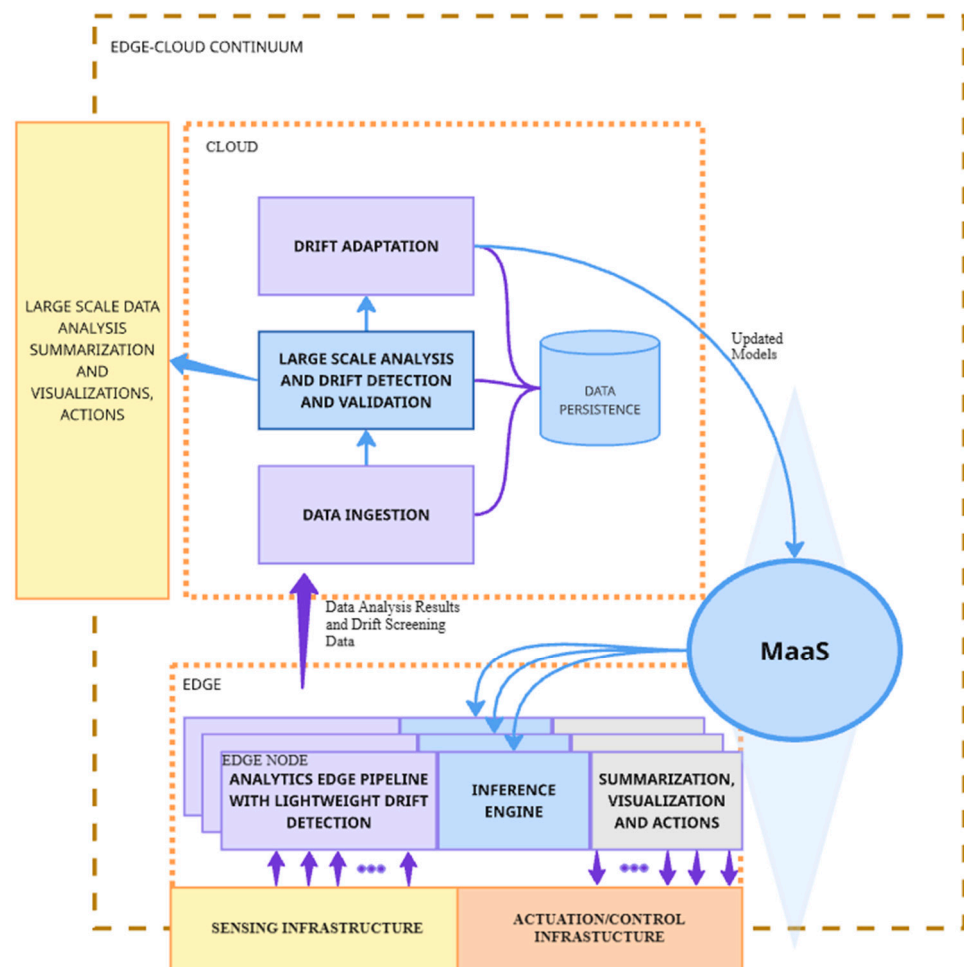


Figure 1. High-level overview of the proposed architecture.

Raw physiological data from the sensing infrastructure are ingested by edge nodes, each associated with an individual user. Each edge node runs an analytics pipeline with lightweight drift detection, an inference engine, and an optional summarization and visualization component. After analysis, processed data and drift signals are sent to the cloud for further analysis. The cloud performs broader data analysis and drift detection, executes adaptation strategies, and stores all information in a persistence component. New models are delivered to edge inference engines through a Model-as-a-Service (MaaS) component. The following subsections describe each layer in detail.

3.1.1. Component Distribution over Edge-Cloud Continuum

The edge tier operates as the interface between sensing infrastructure and the analytics pipeline, directly ingesting high-frequency physiological data streams such as continuous glucose monitoring measurements. Its primary responsibilities include signal preprocessing, feature extraction, and low-latency inference. Models at this tier are compact and computationally efficient, designed for devices with limited memory, processing power, and battery capacity, such as smartphones, embedded gateways, or wearable-adjacent controllers. In addition to inference, the edge tier performs preliminary drift screening. This screening is lightweight and conservative due to device constraints, and it relies on incremental statistical indicators or model-consistency signals that can be computed with bounded memory and processing overhead. By restricting edge-side screening to low-cost computations, the system lowers energy consumption while still detecting early indications of drifts. To support interpretability and monitoring, the architecture includes a user-facing presentation component responsible for generating personalized summaries, visualizations, list of actions, and optional dashboards. This component can also be used for communication with different actuators or any kind of control infrastructure, thus creating not only adaptive but also responsive systems.

The cloud tier serves as the coordination and decision-making core of the architecture. It aggregates drift indicators received from multiple edge devices and applies more computationally intensive drift validation and analysis methods that require longer temporal context or richer feature representations. These operations are typically not applicable at the edge due to resource limitations or incomplete observability. Beyond drift validation, the cloud tier manages the model lifecycle through dedicated services responsible for model retraining, versioning, and deployment orchestration. Model training is decoupled from serving through a Model-as-a-Service (MaaS) abstraction, enabling independent scaling and fault isolation. MaaS is a logical service responsible for model distribution, version management, and deployment coordination. Individual MaaS instances may be physically deployed at different points of the edge–cloud continuum (e.g., cloud, regional, or near-edge), depending on latency, governance, and infrastructural constraints. This flexibility allows the architecture to adapt to diverse deployment environments without altering core system logic. Furthermore, for large-scale systems this component can be organized in a hierarchical manner, with instances spanning across the edge-cloud continuum. In typical deployments, one or more MaaS instances are placed closer to the edge to reduce model distribution latency, while remaining logically coordinated by cloud-level control services. Cloud-level adaptation decisions are explicitly designed to reflect the realistic nature of distributed IoT environments, including intermittent connectivity, delayed data arrival, and partial observability across devices. As a result, adaptation decisions are based on aggregated and validated drift evidence over time, rather than being strictly reactive to individual drift alerts or isolated deviations. Along with deep drift analysis, the cloud tier can perform broad analysis of the data among users, which can be a valuable tool for clinicians conducting longitudinal studies across different groups of users. Furthermore, it

is possible to add a component that will provide needed summarizations, dashboards and analysis results.

The feedback loop closes the adaptive cycle by translating validated cloud-level adaptation decisions into actionable updates at the edge through an asynchronous, multi-stage pipeline. When drift validation indicates that adaptation is required, updated models are trained in the cloud, logged with complete metadata, and made available for deployment through the MaaS service. Model creation, storage, and deployment are intentionally decoupled. The adaptation engine focuses on responding to validated drift events by producing updated models, while the MaaS service handles model distribution, version management, and coordination with edge devices. This design allows each component to evolve independently and scale according to its workload characteristics. The feedback loop incorporates buffering and validation mechanisms intended to reduce instability caused by noisy or transient drift signals. Model deployment proceeds asynchronously, allowing edge devices to continue inference using existing models until updates are safely applied. The feedback loop is designed not only to guarantee stability for inference results but also to reduce the risk of oscillatory behavior while maintaining continuous operation under evolving data distributions. All feedback-driven updates operate exclusively at the system and model-management level and do not constitute clinical evaluations, diagnoses, or therapeutic recommendations.

3.1.2. Drift Detection, Validation, and Adaptation Logic

Rather than relying on a single detection mechanism, the system adopts a tier-aware drift monitoring strategy in which detection methods are selected based on their computational requirements, temporal sensitivity, robustness to noise, and suitability for distributed execution.

At the edge, drift monitoring is constrained by limited memory, processing capacity, and energy resources. Edge-level screening mechanisms therefore support incremental computation, bounded state, and constant or near-constant time complexity per observation. The architecture relies on window-based statistical tests and simple distributional consistency checks that track changes in low-dimensional summary statistics over sliding windows. The edge may also monitor model-consistency signals derived from the inference process itself, such as variations in prediction confidence, residual distributions, or anomaly scores, which can serve as indirect indicators of potential drift while reusing existing computations. Edge-side drift screening is intentionally conservative, particularly in healthcare-oriented deployments, and is restricted to signaling persistent deviations that justify escalation rather than confirming drift.

The cloud tier performs drift validation and interpretation using methods that cannot be applied at the edge due to their data, computational, or memory demands. With access to longer temporal histories, richer feature representations, and elastic compute resources, the cloud can apply more advanced and expressive drift detection techniques, including distributional distance measures, reconstruction-error-based detectors, and model-based approaches that directly assess changes in the learned input–output relationship. Cloud-side detection can also incorporate contextual information unavailable at the edge, such as historical baselines, cross-user statistics, or population-level trends. This wider perspective allows the system to evaluate whether detected drift is bound to a single user or indicative of broader shifts. Beyond detection, the cloud tier can characterize drift in terms of magnitude, persistence, and relevance to downstream tasks, directly informing adaptation decisions.

Adaptation is not triggered directly by individual drift indicators or isolated deviations. Instead, the adaptation logic is formalized through an aggregated drift confidence score computed for each user or user group over a sliding temporal window. A drift is

actionable only if the score exceeds a threshold for k consecutive windows and no adaptation has occurred within the cooldown interval Δt , preventing oscillatory updates from noisy signals.

To further improve model–data alignment while limiting retraining frequency, the system performs user classification as a system-level optimization strategy, maintaining one model ensemble per user group. Each user group corresponds to a shared model ensemble trained on users exhibiting similar data–model alignment characteristics, rather than demographic or clinical similarity. The number of groups can vary over time to accommodate large-scale concept drifts. Rather than assuming static group membership, user states are modeled as dynamic and subject to transition. Validated drift events may indicate that a user’s data no longer conforms to the assumptions underlying their current model configuration. Such indications initiate a structured evaluation under the same persistence and stability constraints applied to other adaptation decisions. The cloud tier evaluates whether reassignment to an alternative user state is more appropriate than retraining from scratch, prioritizing reassignment as a lower-cost and faster adaptation pathway when an existing model ensemble provides better alignment.

State transitions are governed by the same persistence requirements applied to other adaptation decisions. User states are treated as internal system constructs that are reversible and continuously re-evaluated as new data becomes available. They are not exposed as clinical labels and are not intended to inform medical interpretation or decision-making. This direction is deferred to future work and will require close collaboration with medical experts.

3.2. System Implementation

To test the principles presented, a prototype of the proposed system is implemented to support adaptive, drift-aware healthcare monitoring under IoT constraints. As a supporting use case, analysis of CGM data is chosen. The main purpose of the system from users’ perspective is to detect anomalies in readings and create predictions about glucose levels. The system performs 15 min-ahead glucose-level prediction and anomaly detection on minute-level CGM streams. The system is based on a microservice architecture realized as a set of Docker containers, where each container emulates a specific deployment role within the architecture, including edge devices, cloud-side services, messaging infrastructure, and model management components. The containerized setup enables repeatable evaluation of distributed behavior while preserving realistic interaction patterns and communication overheads. The implementation prioritizes modularity, allowing individual components to be replaced without architectural changes. All implementation choices described below reflect a reference prototype intended to demonstrate feasibility and component interaction, rather than an optimized or clinically validated deployment. The implementation specifics are depicted in Figure 2, and every component is described in the sections below.

3.2.1. Edge-Side Implementation

The edge-side implementation supports real-time CGM ingestion, lightweight inference, and local drift screening, with Dockerized nodes emulating smartphones or embedded gateways to enable reproducible evaluation under realistic IoT constraints. The edge tier is designed to minimize latency and bandwidth usage by performing inference and drift detection locally, transmitting only relevant events and summaries to the cloud tier. Each edge node is implemented as a Python 3.14 service designed for deployment on resource-constrained devices and follows a modular design that includes sensor interface, feature extraction, inference engine, drift screening subsystem, visualization, and control

components. Each edge-role container operates independently and performs local inference on incoming data streams.

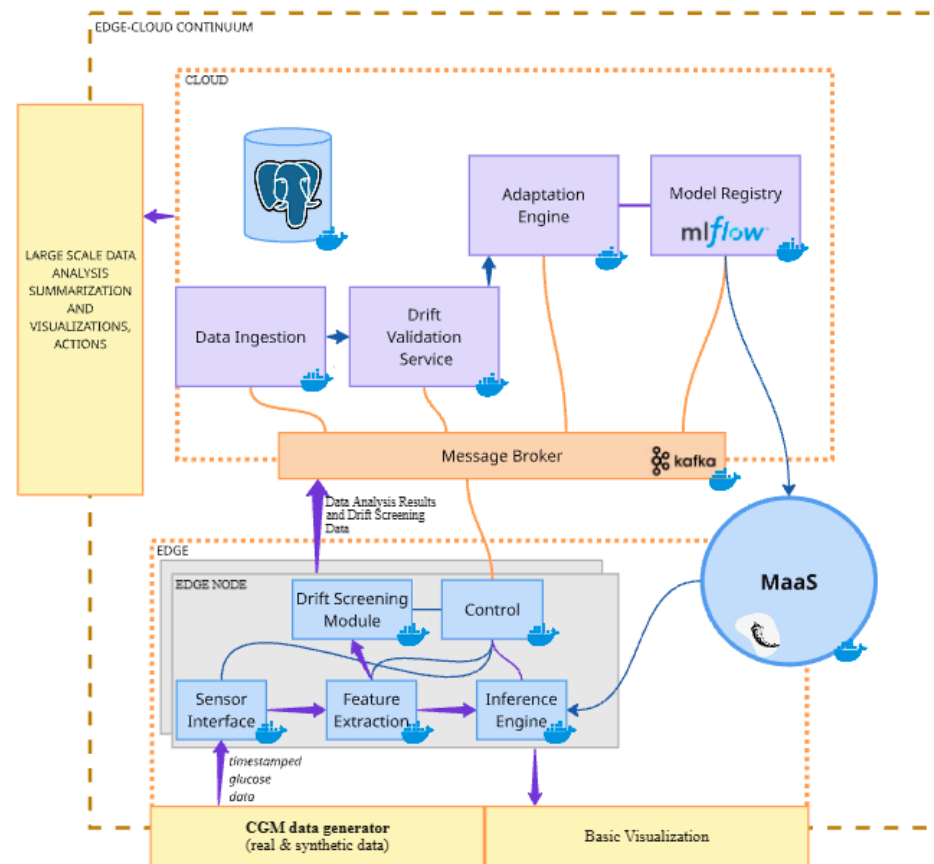


Figure 2. Architecture of the implemented system prototype.

The **sensor interface** component captures the streaming data from real or simulated sources. It supports both real and synthetic data sources. The **feature extraction** component operates on 100-sample sliding windows and computes seven statistical features (mean, standard deviation, minimum, maximum, range, rate of change, and trend) using ring buffers to ensure constant memory usage. The edge **inference engine** performs 15 min-ahead glucose prediction using a lightweight linear regression model. Linear regression was selected for edge deployment due to its minimal computational footprint, bounded memory usage, and constant-time inference cost per sample, making it well suited to resource-constrained devices. The model is trained on the 500 most recent samples using the seven extracted statistical features (Section 3.2.1) as input. The evaluation follows a rolling one-step-ahead protocol in which the model predicts the glucose value 15 min (15 samples) ahead. The architecture supports alternative lightweight models without modification. Models are kept in memory and updated atomically, ensuring uninterrupted operation and consistency during deployment. The **drift screening** subsystem employs a four-detector ensemble to capture complementary drift patterns in CGM data. ADWIN provides sensitivity to gradual drift using adaptive windows with $O(\log N)$ memory complexity, while the Page–Hinkley test keeps a cumulative sum of deviations from the mean. Two additional statistical detectors monitor mean and variance over fixed 100-sample windows, triggering when the mean deviates by more than 2.0 standard deviations or when variance exceeds $1.5 \times$ the baseline. These window sizes and thresholds balance sensitivity to sustained distributional shifts against robustness to short-term postprandial variability. All detectors operate concurrently on each sample, and drift alerts are emitted only when at

least two detectors agree within a 60 s window. This threshold-based consensus strategy improves robustness across gradual and abrupt drift types while limiting false positives. Parameters reflect experimental settings and are not assumed to be universally optimal. Table 2 summarizes the configuration of each detector, including the signal monitored, key hyperparameters, and memory complexity.

Table 2. Drift detector configuration in the edge screening subsystem.

Detector	Signal Monitored	Key Hyperparameters	Memory Complexity
ADWIN	Prediction residuals	$\delta = 0.002$	$O(\log N)$
Page-Hinkley	Prediction residuals	$\lambda = 50, \alpha = 0.005$	$O(1)$
Mean-shift monitor	Feature-window mean (raw CGM)	Window = 100 samples, threshold = 2.0σ	$O(W)$
Variance-ratio monitor	Feature-window mean (raw CGM)	Window = 100 samples, threshold = $1.5 \times \text{baseline}$	$O(W)$

All four detectors operate concurrently on each incoming sample. Drift consensus requires that at least two detectors emit an alarm within a 60 s sliding window. This voting window is evaluated continuously: when two or more detectors register alarms whose timestamps fall within 60 s of each other. The alert confidence score is equal to the fraction of active detectors (e.g., $2/4 = 0.50$, $3/4 = 0.75$, $4/4 = 1.0$).

The **visualization** component is realized for the purpose of the prototype as a lightweight service that periodically generates visualizations of observed and predicted glucose values and model performance.

3.2.2. Cloud-Side Implementation and MaaS Integration

The cloud-side implementation is realized as a microservices architecture that includes following primary services: Drift validation service for statistical confirmation, Adaptation engine for model creation and retraining, Model registry for model-related artifact management, Message broker for asynchronous communication, and Database for metadata persistence. Along with the primary components, a data ingestion component is added for data coordination. Furthermore, cloud components are integrated with the MaaS (Model-as-a-Service) component for model deployment coordination. This modular design enables independent scaling, fault isolation, and incremental evolution of individual components while maintaining a cohesive control plane for drift-driven adaptation.

The separation of concerns is deliberate: the adaptation engine handles all model training operations, including baseline creation and drift-triggered retraining, while the MaaS service focuses exclusively on model serving and deployment to edge devices. This division allows the adaptation engine to operate as a batch-oriented training service that can leverage significant computational resources during retraining, while the MaaS service maintains low-latency responsiveness for edge model requests. Communication between these services flows through MLflow and via a message broker for operational notifications, ensuring loose coupling and independent scalability.

The drift validation service implements a two-stage filtering pipeline that suppresses spurious drift alerts before any adaptation is initiated. In the first stage, incoming drift indicators are screened against a minimum confidence threshold of 0.50; alerts below this threshold are discarded as weak signals. In the second stage, alerts passing the initial screen are classified into severity levels based on their confidence score: LOW (confidence in $[0.50, 0.60)$), MEDIUM ($[0.60, 0.75)$), HIGH ($[0.75, 1.00)$), and CRITICAL (≥ 1.00). Only HIGH and CRITICAL events are considered actionable and are forwarded to the adaptation

decision logic. Each validated alert is enriched with metadata describing the originating detector type, monitored metric, and original confidence score.

Algorithm 1 formalizes the cloud-side drift validation and adaptation triggering logic.

Algorithm 1. Cloud-Side Drift Validation and Adaptation Triggering

Input: drift_alert (user_id, detector_type, confidence, timestamp)

1. Filter: If confidence < $C_{\min}$, discard the alert and return.

2. Classify severity:

- If confidence $\in [C_{\min}, C_{\text{med}})$: severity $\leftarrow$ LOW
- If confidence $\in [C_{\text{med}}, C_{\text{high}})$: severity $\leftarrow$ MEDIUM
- If confidence $\in [C_{\text{high}}, C_{\text{crit}})$: severity $\leftarrow$ HIGH
- If confidence $\geq C_{\text{crit}}$: severity $\leftarrow$ CRITICAL

3. Actionability check: If severity < HIGH, log the alert and return.

4. Persistence check: Verify that severity $\geq$ HIGH has been sustained for at least k consecutive evaluation windows of duration T_w for this user_id.

5. Cooldown check: Verify that no adaptation has been executed for this user_id within the cooldown interval Δt .

6. Decision: If both persistence and cooldown checks pass, emit a validated drift event for adaptation.

Output: validated_drift_event OR suppressed alert

The parameter values used in the prototype evaluation are: $C_{\min} = 0.50$, $C_{\text{med}} = 0.60$, $C_{\text{high}} = 0.75$, $C_{\text{crit}} = 1.00$, $k = 2$ consecutive windows, $T_w = 60$ s, and $\Delta t = 3600$ s.

The adaptation engine handles both initial baseline model creation and drift-triggered retraining. At system initialization, baseline models are generated for all registered users using synthetic physiological data. During runtime, the engine processes validated drift events asynchronously, enforcing per-user stability constraints through cooldown intervals to prevent oscillatory retraining. When adaptation is needed, the engine retrains models using a sliding window of the most recent post-drift-onset observations, excluding data collected before the detected distributional shift to avoid training on outdated patterns. The performance improvements are evaluated, and the complete training context is logged to the registry.

The system integrates MLflow as a centralized model registry. Trained models are organized into per-user experiments, each containing multiple runs corresponding to successive model versions. Artifacts are logged together with comprehensive metadata covering training configuration, performance metrics (MAE, RMSE, R^2), and execution environment specifications. The registry maintains a complete version history per user, supporting temporal analysis and rollback.

The MaaS component acts as the model deployment and distribution layer. While MaaS instances may be deployed near edge devices for latency reasons, their operation remains logically coordinated by cloud-side adaptation and registry services. Operating reactively, it responds to notifications from the adaptation engine, retrieves and validates model artifacts from the registry, and asynchronously issues deployment commands to the appropriate edge nodes. It also exposes a lightweight service interface that allows edge devices to query the latest or best-performing model for a given user. By delegating all training responsibilities to the adaptation engine, MaaS remains stateless and focused on reliable model distribution and integrity validation.

Persistent system state is maintained using PostgreSQL (v15) as a database that provides ACID-compliant persistence for all coordination and adaptation states. The schema includes six primary tables covering user state, model metadata, edge-level drift indicators,

cloud-level drift validations, and executed adaptation actions. User states (e.g., STABLE, TRANSITIONING, HIGH_VARIABILITY, DRIFTING) describe internal model–data alignment and adaptation context rather than clinical conditions. Referential integrity is enforced through foreign key constraints, and indexing strategies support scalable querying over drift and adaptation histories.

3.2.3. Feedback Loop and Real-Time Adaptation

The feedback loop is realized as an asynchronous, event-driven orchestration pipeline that decouples drift detection, validation, retraining, and deployment while preserving consistency and ordered execution. Edge containers perform real-time inference and drift screening continuously. When significant deviations are detected, drift indicators are emitted to the cloud using Kafka topics with metadata including timestamps, detector type, and confidence scores. The cloud-side drift validation service aggregates evidence across time and applies configurable thresholds and persistence criteria. Validation logic tolerates delayed, incomplete, or out-of-order event streams.

Once a drift event is validated, the adaptation engine initiates retraining, and newly trained models are registered in MLflow. The MaaS component then retrieves updated artifacts and distributes deployment commands to the corresponding edge containers. Model deployment proceeds asynchronously, allowing edge devices to continue inference using existing models until updates are safely activated.

The following operational rules govern adaptation behavior. The per-user cooldown interval is set to $\Delta t = 3600$ s, chosen to exceed the typical retraining duration plus a stabilization period during which post-deployment model performance is monitored. The post-drift training window consists of the $N = 500$ most recent observations collected after the estimated drift onset. Observations preceding the onset are excluded to avoid training on outdated distributional patterns. During retraining, the edge continues inference using the currently deployed model. Model replacement is atomic: the new model is swapped into the inference engine in a single operation only after it has been validated against performance criteria in the cloud. The previous model version is retained in the MLflow registry. If the post-deployment MAE of the new model exceeds $1.5\times$ the pre-adaptation baseline within a monitoring window of 100 samples, the system initiates an automatic rollback to the previous version. This mechanism ensures continuous inference availability and provides a safety net against adaptation to spurious drift signals.

Inter-component communication uses a hierarchical topic namespace: edge-to-cloud drift notifications are published on `drift/alerts/{user_id}`, validated events propagate on `drift/validated/{user_id}`, and deployment commands are issued on `models/updates/{user_id}`. Message schemas are formally defined using Pydantic models to ensure type safety across all services. End-to-end latency of the feedback loop is dominated by retraining time and varies with workload characteristics.

4. Experimental Evaluation

4.1. CGM Datasets

We used two datasets: one that simulates glucose readings for multiple users, with different types of drifts injected, and one real-world dataset with readings obtained from a single patient throughout different life phases.

A synthetic dataset was generated to simulate common patterns observed in continuous glucose monitoring, including gradual, abrupt and recurrent drifts. Gradual drift is represented by a slow, incremental change in mean glucose levels over time. Abrupt drift models sudden shifts in glucose distribution and is used to mimic unexpected physiological changes. Recurrent drift is implemented by including periods of increased short-term fluctuation.

tuations. These synthetic streams are created to allow precise evaluation of drift detection performance and feedback-loop response. Glucose readings for 20 users were simulated. Synthetic glucose streams were generated with a base glucose range of 70–180 mg/dL, sampled at one-minute intervals. Independent Gaussian noise with $\sigma = 10$ mg/dL was added to each sample. Gradual drift was injected as a linear shift in mean glucose of +20 mg/dL over 10,000 samples. Abrupt drift was implemented as a step change of +30 mg/dL at a single time point. Recurrent drift was modeled as periodic variance increases of 50% with a cycle length of 1440 samples. No artificial missingness or outlier injection was applied; the synthetic streams are therefore free of the sampling irregularities present in the real CGM data. Synthetic data were used exclusively for testing during development and stress evaluation, and they do not form part of the operational system logic.

The real-world dataset used in this study was obtained from a single user and spans multiple, distinct physiological and treatment-related phases, providing a natural source of data drifts for system evaluation. The data were collected using a CGM device and later replayed through the containerized edge–cloud pipeline to assess system behavior under realistic conditions. The dataset covers four sequential phases. The first phase corresponds to gestational diabetes during pregnancy, characterized by regulated glucose dynamics under heightened physiological variability. The second phase captures the post-pregnancy period, during which glucose patterns shift as pregnancy-related metabolic influences subside. The third phase corresponds to the post-breastfeeding period, introducing additional changes in glucose dynamics associated with altered energy demands and hormonal regulation. The fourth phase reflects the initiation of medication-based management, resulting in further distributional changes in the CGM signal. These phases introduce naturally occurring distributional shifts in mean glucose levels, variability, and temporal structure, without any artificial manipulation of the data. The original CGM recordings contain occasional gaps due to sensor unavailability. For the purposes of system replay and drift evaluation, the dataset was transformed into a continuous data source, in which all available samples were streamed sequentially without explicitly modeling missing intervals. That is, gaps in wall-clock time were ignored, and the replay mechanism operated on the observed glucose measurements as a temporally ordered but uninterrupted stream. The transitions between phases are known, enabling qualitative assessment of alignment between detected drift events and documented physiological or treatment changes. Table 3 summarizes the key characteristics of both evaluation datasets.

Table 3. Key Characteristics of the evaluation datasets.

Property	Synthetic Dataset	Real CGM Dataset
Number of users	20	1
Total samples per user	525,600	864,000
Sampling interval	1 min	1 min
Duration per user	1 year	28 months calendar span; 864,000 observed samples (~70% coverage)
Drift types	Gradual, abrupt, recurrent (injected)	Natural phase transitions
Known drift events per users	Varies across users, 2 to 500.	3 (phase boundaries)
Noise model	Gaussian, with σ that varies across users	Natural sensor noise
Missing values	None	Present

4.2. Experimental Setup and Evaluation Scope

The proposed architecture was evaluated using a containerized edge–cloud testbed emulating a distributed IoT deployment with multiple concurrent users. Each user is

represented by an independent edge-role container executing continuous inference, local drift screening, and asynchronous communication with cloud services. The evaluation includes 20 simulated users with synthetically generated physiological time-series and one user with real continuous glucose monitoring (CGM) data, used to validate system behavior under realistic non-stationary conditions. Synthetic data allows controlled injection of drift events with known onset, magnitude, and duration, while real CGM data introduces irregular sampling, noise, and user-specific variability that cannot be easily modeled. The results focus on system-level performance, including drift propagation, validation accuracy, adaptation timing, feedback-loop latency, and model lifecycle behavior. Prediction accuracy is reported just to illustrate adaptation effectiveness and stability. The evaluation scope is bounded by the containerized testbed and the user population described above; the results are interpreted as evidence of architectural feasibility under controlled conditions rather than as a demonstration of scalability to large populations or clinical readiness.

The system successfully maintained continuous inference for all users throughout the evaluation period. Edge-role containers operated independently, generating drift indicators and inference outputs without centralized coordination, while cloud services aggregated and processed events asynchronously. Figure 3 illustrates the end-to-end execution timeline for three representative users, including one real CGM user.

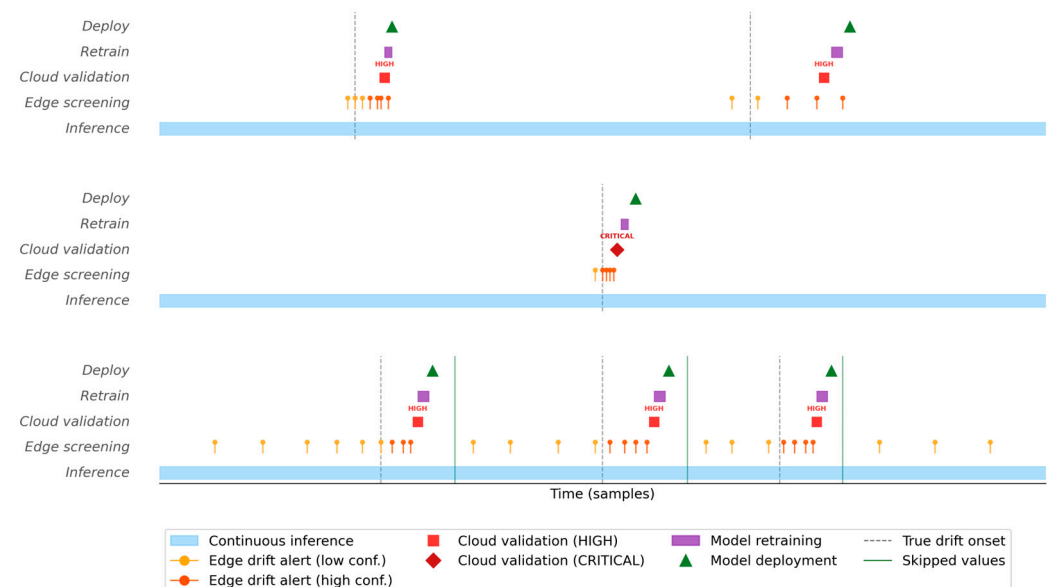


Figure 3. End-to-end execution timeline showing inference, drift screening, validation, retraining, and deployment events for multiple users.

The results demonstrate that inference remains uninterrupted during drift validation, retraining, and model deployment. Updated models are applied only after successful registration and distribution, confirming the effectiveness of decoupled lifecycle management. During the evaluation, the system initiated three user groups and performed two reclassification events. These events are reported to illustrate feasibility and system behavior rather than to provide a quantitative assessment of reclassification optimality.

Edge-level drift screening produced frequent low-confidence alerts in response to short-term variability, particularly for the real CGM user, often coinciding with reported anomalies in the data. However, most of these alerts were filtered out during cloud-side validation. This confirms the intended architectural behavior: high sensitivity at the edge combined with high specificity in the cloud.

Model performance improved following validated adaptation events, particularly for the real CGM user where sustained drift caused baseline degradation. Figure 4 shows MAE over time with (blue line) and without (red line) the model update for the real CGM user, annotated with drift validation (red squares and dotted line) and adaptation events (green triangles and dash-dotted line).

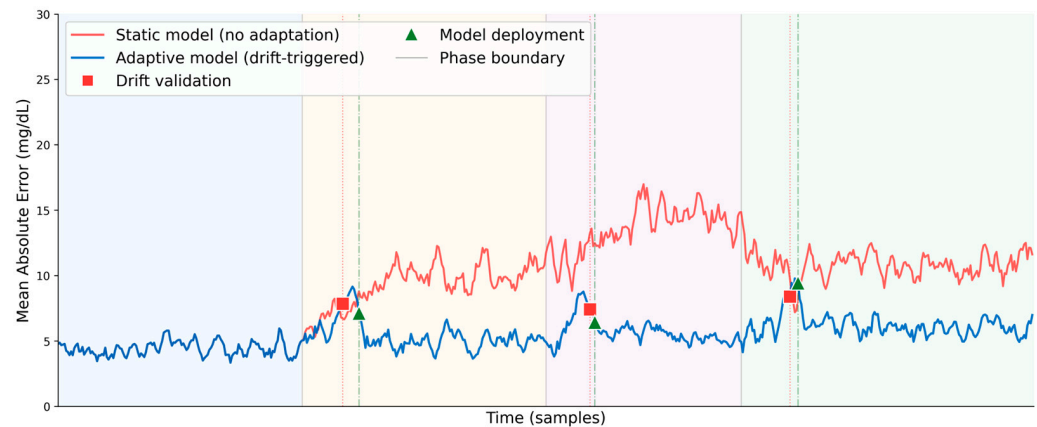


Figure 4. Prediction error (MAE, mg/dL) over time for the real CGM user.

The real CGM user exhibited higher short-term variability and a greater number of edge-level drift alerts compared to simulated users. However, cloud-side validation filtered a large number of these alerts, resulting in adaptation behavior comparable to simulated users with injected drift. This demonstrates that the architecture generalizes across synthetic and real-world physiological data without requiring domain-specific tuning at the edge.

To quantify system-level performance beyond prediction accuracy, we instrumented the prototype to record timestamps at each stage of the drift handling pipeline: edge alert emission, cloud receipt, validation decision, retraining initiation, retraining completion, model registration, and edge deployment confirmation. Detection delay is defined as the interval between the known or estimated drift onset and the first validated cloud-level drift event. End-to-end adaptation latency is measured from drift onset to confirmed edge deployment of the updated model. Cloud validation pass rate is computed as the ratio of cloud-validated drift events to total edge alerts received. The false positive rate is estimated for synthetic users by comparing validated drift events against the known injection schedule; any validated event not corresponding to a known drift within a tolerance window of 200 samples is counted as a false positive. Table 4 reports system-level performance metrics aggregated across all evaluation users. The mean detection delay from drift onset to validated cloud alert was 60.25 ± 35.2 s across synthetic users, reflecting primarily the time required for edge-side detectors to accumulate sufficient statistical evidence, of which cloud-side validation processing accounted for 4 ± 2.2 s, with the remaining ~ 56 s representing edge-side detector accumulation time. The subsequent retraining (5 ± 2 s) and deployment (~ 1 s) stages yield a total end-to-end latency of 66 ± 37 s. Following validation, the adaptation pipeline required 5 ± 2 s for model retraining and registration (dominated by MLflow logging and metadata persistence rather than model fitting, which completed in under 1 ms), and approximately 1 s for deployment to the edge, yielding a total end-to-end latency of approximately 66 ± 37 s from drift onset to the active updated model. Of 300–1800 total edge-level drift alerts per user, $15.32 \pm 5.31\%$ were confirmed by cloud validation, yielding an estimated false positive rate of $27.32 \pm 19.20\%$ relative to known synthetic drift events. While this FPR indicates room for improvement in validation precision, the cooldown mechanism limits consecutive instability; the estimated ~ 23 false-positive retraining events per user correspond to approximately 115 s of cumu-

lative compute. The real CGM user generated approximately $2\times$ more edge-level alerts than the synthetic average, driven by higher short-term postprandial variability. However, the cloud validation pass rate was substantially lower ($\sim 0.25\%$ versus 15.32% for synthetic users), reflecting the greater proportion of noise-driven alerts in real physiological data. The absolute number of validated drift events (~ 5) aligned with the three known phase boundaries, confirming that the cloud filter correctly identified clinically meaningful distributional shifts while suppressing transient variability—consistent with the intended design of high edge sensitivity combined with selective cloud filtering.

Table 4. System-level performance metrics. Detection delay and adaptation latency are reported as mean $\pm$ standard deviation across 20 synthetic users. The real CGM user is reported separately.

Metric	Synthetic Users (Mean $\pm$ SD or Range)	Real CGM User
Total edge alerts	[300–1800]	~ 2000
Cloud validation pass rate (%)	15.32 ± 5.31	0.25
Estimated false positive rate (%)	27.32 ± 19.20	N/A (no ground truth)
Validated drift events	[30–250]	5
Detection delay (s)	60.25 ± 35.2	N/A *
— of which: cloud validation processing (s)	4 ± 2.2	4
Adaptation pipeline—retraining + registration (s)	5 ± 2	~ 5
Adaptation pipeline—deployment (s)	1	~ 1
Total end-to-end latency—onset to deployment (s)	66 ± 37	N/A * (~ 6 s pipeline-only)
Total retraining events	85 ± 34	5

* For the real CGM user, detection delay relative to true onset cannot be computed without ground truth. Alignment with known phase transitions is discussed qualitatively in Section 4.3.

To assess the contribution of individual architectural components, we conducted three ablation experiments on the synthetic dataset (20 users). In the first ablation (edge-only triggering), cloud validation was bypassed and every edge-level consensus alert directly triggered adaptation. In the second ablation (single detector), only ADWIN was retained and the ensemble voting mechanism was disabled; any ADWIN alarm triggered an edge alert. In the third ablation (no cooldown), the per-user cooldown interval was set to zero, allowing consecutive adaptations without a stabilization period. Each ablation was compared against the full pipeline configuration using the same synthetic data streams and drift injection schedule.

Table 5 presents the ablation results. Removing cloud validation (edge-only) increased retraining events by approximately $7\times$ (range: $3.5\times$ – $21\times$) and raised the false positive rate from 27.32% to 88.87% , confirming that cloud-level filtering is essential for suppressing transient alerts. Using a single detector (ADWIN only) increased the false positive rate by 46.1 percentage points relative to the ensemble, while also increasing false negatives: the ensemble detected 5.61 true drifts per user on average compared to 4.90 for ADWIN alone, with the difference attributable primarily to variance-driven drifts captured by the variance-ratio monitor but missed by ADWIN. Disabling the cooldown mechanism resulted in 4.2 ± 1.8 oscillatory retraining episodes per user in which the system retrained up to $4\times$ within 120 s for the same user, with no net improvement in MAE. These results support the design choices of hierarchical validation, multi-detector voting, and cooldown-based stability constraints.

To contextualize the drift-aware approach against simpler adaptation strategies, we implemented two baselines on the synthetic dataset. Baseline A (periodic retraining) retrains each user’s model at fixed intervals of $N = \{500, 2000, 10,000\}$ samples regardless of drift status. Baseline B (performance-triggered retraining) monitors a rolling MAE computed over the most recent 100 samples and initiates retraining whenever MAE exceeds

2.0× the post-training baseline MAE. Both baselines use the same training window size and model configuration as the drift-aware pipeline to ensure comparability. The comparison metrics are mean post-adaptation MAE, total retraining events per user, and cumulative retraining compute time.

Table 5. Ablation analysis across 20 synthetic users. FPR = false positive rate (validated alerts not corresponding to known drift). Retraining events and MAE are reported as mean ± SD per user.

Configuration	FPR (%)	Retraining Events	Post-Adaptation MAE (mg/dL)	Oscillatory Episodes
Full pipeline	27.32 ± 19.20	85 ± 34	3.55 ± 0.82	0
Edge-only (no cloud validation)	88.87 ± 4.52	540 ± 220; 300–1800 without cooldown	5.12 ± 1.45	0
Single detector (ADWIN only)	73.37 ± 11.85	230 ± 80	4.38 ± 1.62	0
No cooldown	68.73 ± 14.30	120 ± 40	4.15 ± 1.28	4.2 ± 1.8

Table 6 compares the drift-aware pipeline against both baselines. Periodic retraining at the shortest interval (N = 500) achieved a mean MAE of 5.98 ± 1.32 mg/dL but required 1050 retraining events per user, 12.4× more than the drift-aware pipeline. Periodic retraining at N = 2000 reduced retraining frequency but produced a substantially higher mean MAE of 21.33 ± 5.97 mg/dL, reflecting the high drift frequency in the synthetic dataset where the model frequently operated with outdated parameters between fixed retraining intervals. Performance-triggered retraining achieved a mean MAE of 6.26 ± 1.50 mg/dL with 72 ± 27 retraining events. While this approach responded effectively to abrupt drift scenarios, it exhibited delayed adaptation under gradual drift, where error accumulation remained below the 2.0× triggering threshold for extended periods. The drift-aware pipeline achieved 3.55 ± 0.82 mg/dL with 85 ± 34 retraining events, representing a 40.6% reduction in MAE relative to the best-performing periodic baseline (N = 500) while using 91.9% fewer retraining events. Compared to performance-triggered retraining, the drift-aware approach achieved 43.3% lower MAE at a comparable number of retraining events, with the improvement attributable primarily to timely detection of gradual drift that remained below the performance-based threshold.

Table 6. Baseline comparison across 20 synthetic users. All values are mean ± SD per user.

Strategy	MAE ± SD (mg/dL)	Retraining Events	Cumulative Retraining Time (s)
Periodic (N = 500)	5.98 ± 1.32	1050	5.250
Periodic (N = 2000)	21.33 ± 5.97	260	1.300
Periodic (N = 10,000)	32.99 ± 8.25	52	260
Performance-triggered	6.26 ± 1.50	72 ± 27	360 ± 135
Drift-aware	3.55 ± 0.82	85 ± 34	425 ± 170

Computational overhead was measured on the containerized testbed to characterize the resource cost of drift-aware operation. The testbed hardware consists of Intel Core i7 and 16 GB RAM; each edge container is limited to 1 CPU core and 512 MB RAM; cloud containers unrestricted. Edge-side measurements include per-sample inference latency and per-sample drift detection overhead, both measured as wall-clock time averaged over the full evaluation run. Cloud-side measurements include retraining wall-clock time per user and model artifact size. Alert bandwidth was estimated from the average alert message size and emission frequency.

Table 7 summarizes the computational overhead. Edge inference latency averaged 0.054 ms per sample, while drift detection added 0.009 ms per sample, representing a 16.8% overhead relative to inference alone. Peak memory usage for the drift screening subsystem was 0.91 MB per edge container. Edge alert messages averaged 118 bytes each, with a mean emission rate of 0.069 alerts per hour per user, corresponding to an estimated bandwidth of 0.0079 KB/hour per user. Cloud retraining required 0.0006 s per event for model fitting; the 5 s adaptation pipeline latency reported in Table 4 is dominated by MLflow artifact logging, metadata persistence, and message broker processing rather than model computation. Model artifacts averaged 0.44 KB in size.

Table 7. Computational overhead. Edge metrics are averaged across all users and the full evaluation period. Cloud metrics are per retraining event.

Component	Metric	Value
Edge inference	Latency per sample (ms)	0.0543
Edge drift detection	Overhead per sample (ms)	0.0091
	Peak Memory (MB)	0.9126
Edge alerts	Message size (bytes)	118
	Emission rate (alerts/hour/user)	0.069
	Estimated bandwidth (KB/hour/user)	0.0079
Cloud retraining	Model fitting time per event (s)	0.0006
	Full pipeline time per event (s)	4.8 ± 1.9
Cloud model artifact	Size (KB)	0.44

To assess the sensitivity of drift detection to window parameter choices, we varied the feature extraction window size across {50, 100, 200} samples and the detector voting window across {30, 60, 120} seconds. This yielded nine parameter combinations, evaluated on a subset of five synthetic users selected to include at least one instance of each drift type (gradual, abrupt, recurrent). Detection delay, false positive rate, and number of unnecessary retraining events were recorded for each configuration.

Table 8 reports the sensitivity analysis results. Reducing the feature window to 50 samples increased the false positive rate by 12.5 percentage points relative to the default (100 samples) at a matched voting window of 60 s, as the shorter window made mean-shift and variance-ratio monitors more reactive to transient postprandial variability. Conversely, increasing the feature window to 200 samples raised the mean detection delay by 33.2 s, reflecting the time required for the larger window to accumulate sufficient post-drift samples. For the voting window parameter, a 120 s window increased edge alert frequency by approximately 35% relative to the default 60 s window due to increased coincidental detector agreement, though cloud-side validation absorbed most of the additional alerts without substantially changing the number of unnecessary retraining events (2.6 vs. 2.1). A 30 s voting window reduced edge alert frequency by approximately 24% and lowered the false positive rate to 21.8%, but marginally increased detection delay for gradual drifts requiring sequential detector activation. The default configuration (100-sample feature window, 60 s voting window) represents a balanced trade-off between sensitivity and stability, achieving moderate detection delay (60.2 s) and a false positive rate (27.3%) without favoring either extreme. These parameter choices are not claimed to be universally optimal and may require adjustment for data streams with different temporal characteristics.

Table 8. Window parameter sensitivity analysis across 20 synthetic users. FPR = false positive rate at the cloud validation level.

Feature Window (Samples)	Voting Window (s)	Detection Delay (s)	FPR (%)	Unnecessary Retraining Events
50	30	53.2	34.3	2.6
50	60	45.2	39.8	3.1
50	120	40.2	50.8	3.9
100	30	68.2	21.8	1.7
100	60	60.2	27.3	2.1
100	120	55.2	34.3	2.6
200	30	101.4	10.8	0.8
200	60	93.4	18.8	1.4
200	120	88.4	25.8	2.0

Bold row indicates the default configuration used throughout the evaluation.

4.3. Results and Discussion

The presented results highlight several architectural implications that extend beyond the specific experimental setup. First, the clear separation between edge-level drift screening and cloud-level drift validation proved essential for maintaining system stability. While edge nodes generated a high volume of low-confidence drift alerts, especially for real CGM data characterized by short-term variability, the cloud validation layer consistently filtered transient signals. This confirms that treating concept drift as a hierarchical, system-level event rather than a local trigger is critical for avoiding unnecessary adaptation in distributed IoT environments.

Second, the results demonstrate that asynchronous adaptation is sufficient to preserve continuous inference, even under sustained drift. By decoupling inference, retraining, and deployment, the system avoids inference downtime and reduces operational risk during model updates. The observed feedback-loop behavior shows that adaptation latency is dominated by retraining rather than coordination or communication overhead, suggesting that further optimizations should focus on training efficiency rather than messaging infrastructure.

Third, dynamic user-state modeling emerged as an effective mechanism for limiting retraining frequency. User reclassification enabled the system to respond to moderate drift by reassigning users to existing model ensembles instead of triggering retraining, reducing computational cost while maintaining model–data alignment. Importantly, user states function strictly as internal system abstractions and do not correspond to clinical conditions, preserving a clear separation between system adaptation and medical interpretation. The two reclassification events observed during evaluation illustrate that the mechanism is operationally viable within the prototype; however, quantitative evaluation of reclassification optimality—including comparison of alternative similarity metrics and group configurations—is deferred to future work as a distinct research question.

The inclusion of real CGM data further illustrates the robustness of the architecture. Although real-world data introduced higher variability and irregular sampling compared to synthetic streams, the system exhibited comparable adaptation behavior without requiring domain-specific tuning at the edge. While precise detection delay cannot be computed for the real CGM user due to the absence of exact drift onset timestamps, visual inspection of Figure 4 indicates that all five cloud-validated drift events cluster near the documented phase boundaries, with no validated events occurring during periods of known distributional stability. The pipeline-only latency (cloud validation through deployment) for the real CGM user was approximately 6 s, consistent with synthetic user measurements. This

qualitative alignment, combined with the strict cloud filtering that reduced approximately 2000 edge alerts to only 5 validated events, provides supporting evidence that the hierarchical validation correctly discriminates between physiologically meaningful drift and transient variability. This suggests that the proposed design generalizes across heterogeneous physiological signals and can accommodate realistic noise patterns encountered in practice.

Nevertheless, several limitations should be acknowledged. The evaluation was conducted in a containerized environment that emulates, but does not fully replicate, real-world network instability, device failures, or large-scale population heterogeneity. Additionally, while CGM data provides a representative example of non-stationary physiological signals, the results should be interpreted as system-level validation rather than evidence of clinical performance or safety. While user reclassification reduced retraining frequency in the evaluated scenarios, this work does not provide a comparative quantitative analysis of reclassification versus retraining strategies. The primary goal is to demonstrate architectural feasibility and decision integration; systematic evaluation of reclassification policies and similarity metrics is left for future work. Addressing these limitations will require extended deployments, larger user populations, and interdisciplinary collaboration, particularly when defining clinically informed validation and adaptation constraints.

Overall, the integrated results and discussion support the central premise of this work: that concept drift can be elevated from a passive monitoring signal to an active coordination mechanism governing model lifecycle, deployment, and user-state alignment across an edge–cloud continuum. Within the evaluated scope, the proposed architecture demonstrates a viable foundation for adaptive IoT systems operating under non-stationary data conditions. Validation at larger user populations, under real network instability, and with clinically informed evaluation protocols remains necessary before broader deployment claims can be made.

5. Conclusions

This paper presented a distributed edge–cloud architecture for continuous inference over non-stationary physiological data streams in which concept drift is treated as a first-class system-level supervisory signal. Rather than handling drift as a passive diagnostic or isolated model maintenance event, the proposed design integrates drift detection, validation, and adaptation into a coordinated control loop governing model lifecycle transitions, deployment orchestration, and dynamic user–model alignment across the edge–cloud continuum.

Through a containerized multi-user evaluation incorporating both synthetic data and real continuous glucose monitoring (CGM) data, preliminary results indicate that the architecture can maintain uninterrupted inference while responding selectively to sustained drift. Hierarchical drift handling that combines sensitive edge-level screening with robust cloud-level validation effectively filtered transient variability and prevented unnecessary or oscillatory model adaptation. Asynchronous retraining and deployment preserved system responsiveness, while dynamic user-state modeling reduced retraining frequency by enabling efficient reassignment to existing model ensembles when appropriate.

Preliminary results suggest that the proposed approach generalizes across heterogeneous data conditions without requiring domain-specific tuning at the edge, supporting deployment in realistic IoT environments characterized by non-stationarity, partial observability, and resource constraints, within the scope of the evaluated containerized testbed. Furthermore, all adaptation decisions operate strictly at the system and model-management level, preserving a clear separation between technical adaptation mechanisms and clinical interpretation.

While the current evaluation focuses on system-level feasibility within a controlled containerized environment, the architecture provides a strong foundation for future extensions. These include larger-scale deployments, longer evaluation horizons, and interdisciplinary collaboration to define clinically informed validation protocols and safety constraints. More broadly, the presented work demonstrates how elevating concept drift to a coordination signal can enable robust, adaptive behavior in distributed AI systems, offering a principled pathway toward long-term, real-world operation under evolving data distributions.

Author Contributions: Conceptualization, A.S.I. and D.S.; methodology, A.S.I., M.I. and N.S.; software, A.S.I. and M.I.; investigation A.S.I., N.S. and D.S.; resources, A.S.I. and N.S.; writing—original draft preparation, A.S.I. and D.S.; writing—review and editing, A.S.I., M.I., N.S. and D.S.; validation A.S.I. and M.I.; project administration, D.S. All authors have read and agreed to the published version of the manuscript.

Funding: This work has been supported by the Ministry of Science, Technological Development and Innovation of the Republic of Serbia [Grant Number: 451-03-34/2026-03/200102].

Data Availability Statement: Due to privacy concerns, dataset used for evaluation is available upon request.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CGM Continuous Glucose Monitoring
MaaS Model-as-a-Service

References

1. Ranjani, R.; Naresh, R. Intelligent edge computing for real-time patient health monitoring using adaptive CNN-LSTM. In *Proceedings of the 2025 International Conference on Modern Sustainable Systems (CMSS), Shah Alam, Malaysia, 12–14 August 2025*; IEEE: Piscataway, NJ, USA, 2025. [\[CrossRef\]](#)
2. Lima, R.A.G.; de Macedo, D.D.J.; Lima, C.A.G. Blockchain-enabled distributed data management for smart health: Enhancing integrity and auditability in edge-fog-cloud ecosystems. In *Proceedings of the 2025 IEEE International Conference on Edge Computing and Communications (EDGE), Helsinki, Finland, 7–12 July 2025*; IEEE: Piscataway, NJ, USA, 2025. [\[CrossRef\]](#)
3. Anitha, M.; Patwari, M.; Makadia, K.D.; Saini, D. AI-powered health diagnosis system: Fog-edge enabled unified framework for real-time disease prediction and healthcare monitoring. In *Proceedings of the 2025 International Conference on Communication, Computer, and Information Technology (IC3IT), Mandya, India, 24–25 October 2025*; IEEE: Piscataway, NJ, USA, 2025. [\[CrossRef\]](#)
4. Badidi, E. Edge AI for Early Detection of Chronic Diseases and the Spread of Infectious Diseases: Opportunities, Challenges, and Future Directions. *Future Internet* **2023**, *15*, 370. [\[CrossRef\]](#)
5. Khan, M.M.; Alkhathami, M. Anomaly detection in IoT-based healthcare: Machine learning for enhanced security. *Sci. Rep.* **2024**, *14*, 5872. [\[CrossRef\]](#) [\[PubMed\]](#)
6. Webb, G.I.; Hyde, R.; Cao, H.; Nguyen, H.L.; Petitjean, F. Characterizing concept drift. *arXiv* **2016**, arXiv:1511.03816. [\[CrossRef\]](#)
7. Rahmani, K.; Thapa, R.; Tsou, P.; Chetty, S.C.; Barnes, G.; Lam, C.; Tso, C.F. Assessing the effects of data drift on the performance of machine learning models used in clinical sepsis prediction. *Int. J. Med. Inform.* **2023**, *173*, 104930. [\[CrossRef\]](#) [\[PubMed\]](#)
8. Iwashita, A.S.; Papa, J.P. An overview on concept drift learning. *IEEE Access* **2018**, *7*, 1532–1547. [\[CrossRef\]](#)
9. Abdullahi, M.; Alhussian, H.; Aziz, N.; Abdulkadir, S.J.; Baashar, Y.; Alashhab, A.A.; Afrin, A. A systematic literature review of concept drift mitigation in time-series applications. *IEEE Access* **2025**, *13*, 119380–119410. [\[CrossRef\]](#)
10. Abdullah, S.; Murnane, E.L.; Matthews, M.; Choudhury, T. Circadian computing: Sensing, modeling, and maintaining biological rhythms. In *Mobile Health: Sensors, Analytic Methods, and Applications*; Rehag, J.M., Murphy, S.A., Kumar, S., Eds.; Springer: Cham, Switzerland, 2017; pp. 35–58. [\[CrossRef\]](#)
11. Beten, A.; Lococco, L.; Baig, A.; Karunarathna, T. Systematic analysis of distribution shifts in cross-subject glucose prediction using wearable physiological data. *Eng. Proc.* **2025**, *118*, 88. [\[CrossRef\]](#)
12. Desale, K.S.; Shinde, S.V. ECG signal processing using an optimized sliding window approach for concept drift detection and adaptation. *J. Eng. Sci. Technol. Rev.* **2023**, *16*, 5. [\[CrossRef\]](#)

13. Zertal, S.; Saighi, A.; Kouah, S.; Meshoul, S.; Laboudi, Z. A real-time deep learning approach for electrocardiogram-based cardiovascular disease prediction with adaptive drift detection and generative feature replay. *CMES—Comput. Model. Eng. Sci.* **2025**, *144*, 3737. [\[CrossRef\]](#)
14. Kelly, C.J.; Karthikesalingam, A.; Suleyman, M.; Corrado, G.; King, D. Key challenges for delivering clinical impact with artificial intelligence. *BMC Med.* **2019**, *17*, 195. [\[CrossRef\]](#) [\[PubMed\]](#)
15. Lu, J.; Liu, A.; Dong, F.; Gu, F.; Gama, J.; Zhang, G. Learning under concept drift: A review. *IEEE Trans. Knowl. Data Eng.* **2020**, *32*, 2587–2606. [\[CrossRef\]](#)
16. Yang, L.; Shami, A. A lightweight concept drift detection and adaptation framework for IoT data streams. *IEEE Internet Things Mag.* **2021**, *4*, 96–101. [\[CrossRef\]](#)
17. Desale, K.S.; Shinde, S.V. Concept drift detection and adaption framework using optimized deep learning and adaptive sliding window approach. *Expert Syst.* **2023**, *40*, e13394. [\[CrossRef\]](#)
18. Pratyusha, M.G.; Sinha, A.K. Managing concept drift in IoT health data streams: A dynamic adaptive weighted ensemble approach. In *Smart Embedded Systems*; Sinha, A., Sharma, A., Melek, P.L.A., Caviglia, D., Eds.; CRC Press: Boca Raton, FL, USA, 2023; pp. 233–252.
19. Patil, D.D.; Mudkanna, J.G.; Rokade, D.; Wadhai, V.M. Concept adapting real-time data stream mining for health care applications. In *Advances in Computer Science, Engineering & Applications: Proceedings of the Second International Conference on Computer Science, Engineering and Applications (ICCSEA 2012)*; Springer: Berlin/Heidelberg, Germany, 2012; Volume 1, pp. 341–351. [\[CrossRef\]](#)
20. Prashanth, B.S.; Kumar, M.V.M.; Puneetha, B.H.; Al Muraqab, N.; Hoque, A.; Moonesar, I.A.; Rao, A. Adaptive buffering strategies for incremental learning under concept drift in lifestyle disease modeling. *IEEE Access* **2025**, *13*, 174001–174033. [\[CrossRef\]](#)
21. Zuo, J.; Zeitouni, K.; Taher, Y. Incremental and adaptive feature exploration over time series stream. In *Proceedings of the 2019 IEEE International Conference on Big Data (Big Data), Los Angeles, CA, USA, 9–12 December 2019*; IEEE: Piscataway, NJ, USA, 2019; pp. 593–602. [\[CrossRef\]](#)
22. Toor, A.A.; Usman, M.; Younas, F.; Fong, A.C.M.; Khan, S.A.; Fong, S. Mining massive e-health data streams for IoMT enabled healthcare systems. *Sensors* **2020**, *20*, 2131. [\[CrossRef\]](#) [\[PubMed\]](#)
23. Gkonis, P.; Giannopoulos, A.; Trakadas, P.; Masip-Bruin, X.; D’Andria, F. A survey on IoT-edge-cloud continuum systems: Status, challenges, use cases, and open issues. *Future Internet* **2023**, *15*, 383. [\[CrossRef\]](#)
24. Stojnev Ilić, A.; Stojanović, D. A scalable edge-cloud architecture for smart adaptive streaming data analysis. *Facta Univ. Ser. Autom. Control Robot.* **2025**, *24*, 129–146. [\[CrossRef\]](#)
25. Lukats, D.; Zielinski, O.; Hahn, A.; Stahl, F. A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams. *Int. J. Data Sci. Anal.* **2025**, *19*, 1–31. [\[CrossRef\]](#)
26. Cao, L.; Huo, T.; Li, S.; Zhang, X.; Chen, Y.; Lin, G.; Wu, F.; Ling, Y.; Zhou, Y.; Xie, Q. Cost optimization in edge computing: A survey. *Artif. Intell. Rev.* **2024**, *57*, 312. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.